MODEL

# WEIGHTS · GRADIENTS LEARNING RATE

A comprehensive visual guide to how neural networks learn — from prediction error to gradient calculation to parameter update.




**Core idea: weights are what the model learns; gradients tell how the loss responds to each weight; the learning rate controls the size of the correction.**

# LEARNING MAP Contents

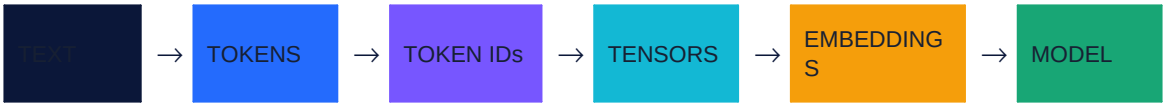
1. Foundation: tensor, model and parameters
2. What model.parameters() actually returns
3. Weight and bias: persistent learnable state
4. Loss: measuring how wrong the prediction is
5. Why loss alone cannot update a weight
6. Gradient: direction + sensitivity
7. Positive, negative and near-zero gradients
8. Backpropagation: how gradients are calculated
9. Learning rate: controlling correction size
10. Optimizer: who actually changes the weights
11. One complete training iteration
12. Multi-weight models: one gradient per scalar parameter
13. PyTorch mapping: .grad, backward(), step(), zero_grad()
14. Worked examples and learning-rate behavior
15. Common misconceptions and debugging
16. Quick-reference cheat sheet

**This revision intentionally preserves the step-by-step reasoning flows used in the conversation instead of compressing them into a single summary diagram.**

# 01 Foundation: tensor → model → parameters

A **tensor** is a container of numbers arranged in zero, one, two or more dimensions. In PyTorch, model inputs, embeddings, weights, biases, activations and gradients are all represented using tensors.

| Structure | Example | Typical shape |
|---|---|---|
| Scalar | 5 | [] |
| Vector | [2, 5, 8] | [3] |
| Matrix | [[1,2,3],[4,5,6]] | [2,3] |
| Token embeddings | 5 tokens × 32 values | [5,32] |




The model itself contains learnable tensors called **parameters**. Most familiar parameters are weights and biases.

# 02 What does model.parameters() mean?

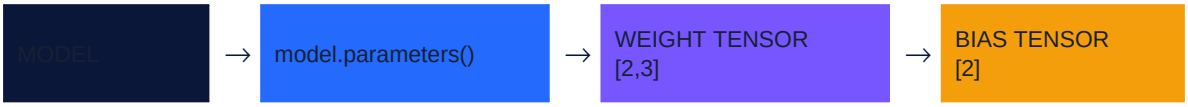
**model.parameters()** returns an iterator over the trainable parameter tensors registered inside a PyTorch model. These are the values an optimizer is normally allowed to update.

```
import torch.nn as nn

model = nn.Linear(3, 2)

for param in model.parameters():
    print(param.shape)

# torch.Size([2, 3])   <- weight
# torch.Size([2])      <- bias
```




With **named_parameters()**, PyTorch also gives the parameter names. This is useful when inspecting a larger network.

```
for name, param in model.named_parameters():
    print(name, param.shape)

# weight torch.Size([2, 3])
# bias   torch.Size([2])
```

**Important: model.parameters() does not contain the training data or the model architecture. It provides access to the learnable parameter tensors stored by the architecture.**

# 03 Weights and biases: what actually gets learned

A **weight** controls the strength of a learned transformation. A **bias** provides an additional learned offset. During training, these parameter values persist and gradually change.




**Weight = what the model currently knows.**
**Gradient = information describing the correction that is needed.**

The gradient is not a second copy of the weight. It is temporary derivative information associated with that weight for the current backward pass.

# 04 Loss: first measure the error

The model first performs a forward pass and makes a prediction. A loss function compares that prediction with the expected target and produces a scalar error value.




A loss value such as **2.4** tells us that the current prediction has error according to the chosen objective. But it does **not** tell a particular weight whether it should become 0.49, 0.51 or something else.

# 05 Why loss alone is not enough to change a weight

Suppose the model currently contains one weight:

**weight = 0.50**




Now we know the model is wrong. But we still have two unanswered questions:

**Should the weight increase?**
0.50 → 0.51 ?

**Or should it decrease?**
0.50 → 0.49 ?

**This is exactly the problem the gradient solves: it tells us how the loss changes when the weight changes slightly.**

# 06 Gradient: direction and sensitivity

For one weight **w**, the gradient is the derivative of loss with respect to that weight:

**gradient = ∂Loss / ∂w**

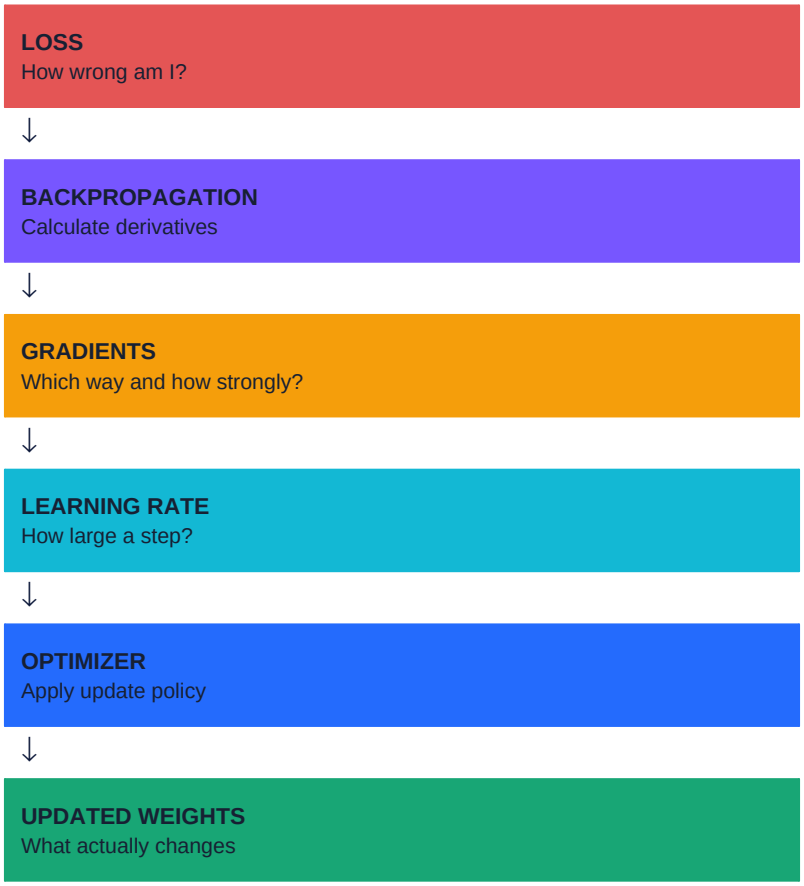
Interpretation: **If I change this weight slightly, in which direction will the loss move, and how sensitive is the loss to that change?**




| Gradient | Meaning near current weight | Basic gradient-descent response |
|---|---|---|
| +2.0 | Increasing weight tends to increase loss | Move weight downward |
| -2.0 | Increasing weight tends to decrease loss | Move weight upward |
| ≈ 0 | Loss is locally less sensitive | Small or no update from this gradient |

# 07 The missing flow: from loss to a weight correction

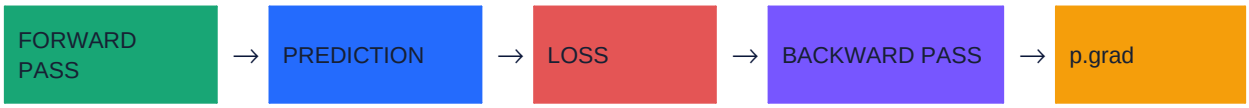
This is the visual chain that connects the concepts step by step:




**Loss measures error → backpropagation calculates gradients → gradients provide correction information → learning rate scales the step → optimizer changes the weights.**

# 08 Backpropagation: where gradients come from

A neural network is a chain of mathematical operations. Backpropagation applies the calculus chain rule from the final loss backward through that computation graph. It calculates a gradient for every trainable parameter that influenced the loss.




```
prediction = model(x)
loss = loss_fn(prediction, target)

loss.backward()     # calculate / accumulate gradients

for name, p in model.named_parameters():
    print(name, p.grad)
```

**loss.backward() does NOT update the weights. It calculates and accumulates gradients. The optimizer performs the actual parameter update.**

# 09 Learning rate: control the size of the correction

Even when the gradient tells us the direction and sensitivity, we usually do not move the weight by the full gradient amount. The **learning rate** scales the update.

**w_new = w_old – learning_rate × gradient**

| Too small | Useful range | Too large |
|---|---|---|
| Tiny steps<br>slow learning | Stable progress toward lower loss | Overshoot<br>oscillation / instability |

### Worked example

```
old weight    = 0.50
gradient      = +2.0
learning rate = 0.01

new weight
= 0.50 - (0.01 × 2.0)
= 0.50 - 0.02
= 0.48
```

# 10 Optimizer: the component that actually changes parameters

The optimizer receives the model parameters, reads their gradients after backpropagation, and applies an update rule when **optimizer.step()** is called.



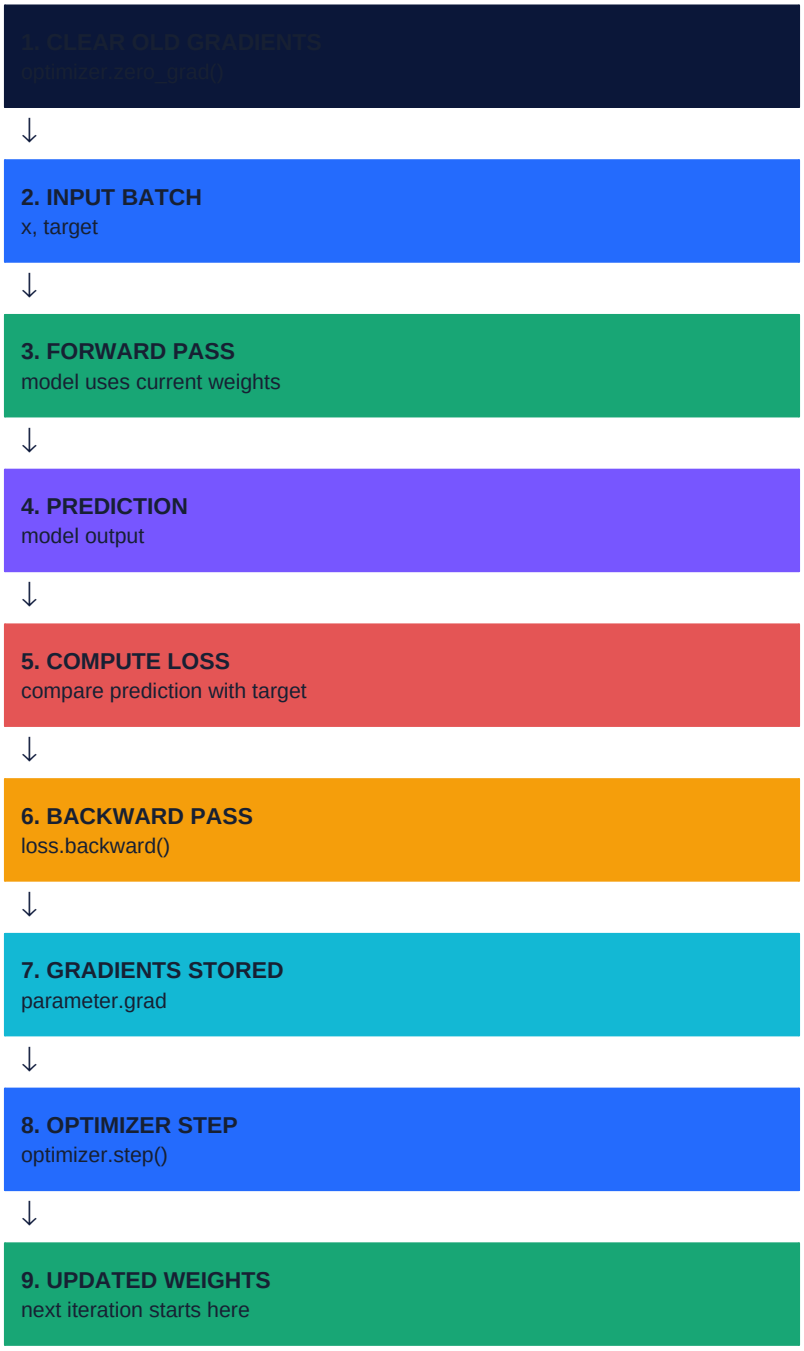

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

optimizer.zero_grad()
prediction = model(x)
loss = loss_fn(prediction, target)
loss.backward()
optimizer.step()
```

**zero_grad() clears old gradients. backward() calculates new gradients. step() uses those gradients to update the parameters.**

# 11 The complete training loop — forward and backward

This is the full flow, including the two distinct phases:



# 12 What happens when there are millions of weights?

Real models contain many parameter tensors and potentially millions or billions of scalar parameter values. Backpropagation calculates a corresponding gradient for each trainable scalar parameter.

| Weight | Gradient | Interpretation |
|---|---|---|
| 0.21 | +0.034 | small downward pressure in basic GD |
| -0.73 | -0.002 | very small upward pressure |
| 0.14 | +0.910 | much stronger local sensitivity |
| 0.82 | -0.0001 | almost locally flat |




A parameter tensor and its gradient tensor normally have the same shape. For example, a weight tensor of shape **[2,3]** normally has a gradient tensor of shape **[2,3]**.

# 13 PyTorch mapping: see the weights and gradients directly

```
for name, p in model.named_parameters():
    print("NAME:", name)
    print("PARAMETER:", p.data)
    print("GRADIENT:", p.grad)
```

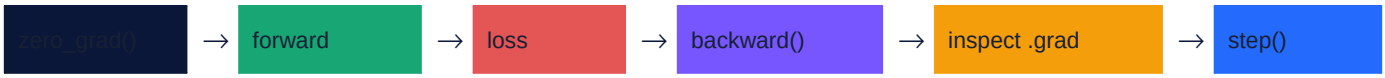
Before **loss.backward()**, **p.grad** may be None (or may contain an earlier accumulated gradient). After backward, trainable parameters that participated in the graph normally have gradient tensors.

```
optimizer.zero_grad()

loss = loss_fn(model(x), target)
loss.backward()

# inspect BEFORE the update
for name, p in model.named_parameters():
    print(name, "param=", p.data, "grad=", p.grad)

optimizer.step()
```



# 14 One-weight example: see every concept in one place

| Stage | Value / question | Role |
|---|---|---|
| Current weight | 0.50 | Current learned value |
| Prediction | Model output | Forward pass result |
| Loss | 2.4 | How wrong? |
| Gradient | +2.0 | Direction + local sensitivity |
| Learning rate | 0.01 | Step-size control |
| Update | –0.02 | –lr × gradient |
| New weight | 0.48 | Parameter after optimizer step |



# 15 Common misconceptions and debugging

**“Gradient is another weight.”**
**No. Weight is persistent model state; gradient is derivative information.**

**“loss.backward() changes weights.”**
**No. It calculates/accumulates gradients.**

**“Loss tells each weight exactly what to do.”**
**Not directly. Differentiation creates per-parameter gradients.**

**“Large gradient means use that exact value as update.”**
**No. Learning rate and optimizer transform it into an update.**

**“Gradients replace themselves automatically.”**
**PyTorch gradients accumulate by default; clear them intentionally.**

**“All parameters must train.”**
**Parameters can be frozen with requires_grad=False.**

### Debugging checklist

| Symptom | Inspect |
|---|---|
| Loss not decreasing | learning rate, data/targets, gradient magnitude, optimizer setup |
| p.grad is None | requires_grad, detached graph, whether backward ran |
| No weight change | optimizer parameters, step() call, nonzero gradients |
| Training unstable | learning rate, batch size, initialization, clipping/normalization |

# 16 Quick-reference cheat sheet

| Concept | Meaning | PyTorch |
|---|---|---|
| Tensor | N-dimensional container of numbers | torch.Tensor |
| Parameter | Learnable tensor registered in model | model.parameters() |
| Weight | Learned transformation strength | layer.weight |
| Bias | Learned additive offset | layer.bias |
| Loss | Scalar measure of prediction error | loss_fn(...) |
| Gradient | Derivative of loss w.r.t. parameter | parameter.grad |
| Backpropagation | Calculates gradients through graph | loss.backward() |
| Learning rate | Controls update scale | lr=... |
| Optimizer | Turns gradient information into updates | torch.optim.Adam(...) |
| Clear gradients | Reset accumulated gradient buffers | optimizer.zero_grad() |
| Update weights | Apply optimizer policy | optimizer.step() |

**Final mental model**
**Input → current weights → prediction → loss → backward → gradients → optimizer + learning rate → updated weights → repeat.**

**Remember: the weight is the value being learned. The gradient is the information that tells training how changing that value affects the loss.**